

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Ingegneria del Software: Tecniche Avanzate

Progetto ISTA – Evoluzione di CodeSmile

Impact Analysis

Professore:

**Ch.mo Prof.
Andrea De Lucia**

Tutor di riferimento:

**Giammaria Giordano
Gilberto Recupito**

Studenti:

**Antonio D'Auria
Mat. NF22500063
Emanuele D'Auria
Mat. 0522501918
Luca Casillo
Mat. NF22500037**

ANNO ACCADEMICO 2025/2026

INDICE

Indice		
1	Introduzione	1
1.1	Obiettivo e Campo di Applicazione	1
1.2	Unità di Analisi	1
1.3	Insiemi di Impatto	1
1.4	Metriche di Accuratezza	2
2	Modello Statico delle Dipendenze	3
2.1	Static Analysis Tool	3
2.2	Flusso Statico della Web Application	3
2.3	Configurazione dei Servizi	4
2.4	Regole di Derivazione	4
3	Impact Analysis di CR1 e CR2	6
3.1	CR1 – Generazione del Call Graph	6
3.1.1	SIS e CIS	6
3.1.2	AIS, FPIS e DIS	6
3.2	CR2 – Estensione della CLI con Opzioni Avanzate	6
3.2.1	SIS e CIS	6
3.2.2	AIS, FPIS e DIS	7
4	Impact Analysis di CR3 e CR4	8
4.1	CR3 – Portabilità tra Ambiente Locale e Docker	8
4.1.1	SIS e CIS	8
4.1.2	AIS, FPIS e DIS	8
4.2	CR4 – Visualizzazione del Call Graph nella WebApp	9
4.2.1	SIS e CIS	9
4.2.2	AIS, FPIS e DIS	9
4.3	Valutazione Complessiva	10

1 Introduzione

Il presente documento applica l'*Impact Analysis* alle quattro Change Request definite per l'evoluzione di CodeSmile. Lo scopo consiste nello stabilire, con un procedimento verificabile, quali unità del sistema potevano essere coinvolte e quali sono state effettivamente interessate durante l'implementazione.

L'analisi è condotta in due momenti. In primo luogo, la struttura descritta nel Reverse Engineering e le dipendenze osservabili nel codice pre-modifica sono utilizzate per ricavare gli insiemi di impatto atteso. In secondo luogo, tali insiemi sono confrontati con le unità realmente modificate, così da identificare previsioni eccessive, dipendenze scoperte in corso d'opera e accuratezza complessiva dell'analisi.

1.1 Obiettivo e Campo di Applicazione

Gli obiettivi del documento sono:

- rendere esplicito il passaggio dalle dipendenze statiche agli insiemi di impatto;
- distinguere le unità direttamente coinvolte dalle candidate raggiunte per propagazione;
- confrontare impatto previsto e impatto effettivo per ciascuna Change Request;
- misurare la qualità della previsione mediante Precision e Recall;
- fornire al Post-Modification Testing un perimetro di verifica motivato, senza anticiparne esecuzione ed esiti.

Il campo di applicazione comprende lo Static Analysis Tool, la CLI, i servizi backend coinvolti nel flusso statico e la Web Application. La semantica del componente AI-Based, i modelli impiegati e la qualità delle relative predizioni rimangono fuori scope. La sua compatibilità infrastrutturale costituisce esclusivamente un vincolo di non-regressione da verificare nel documento di testing post-modifica.

1.2 Unità di Analisi

La granularità adottata è il file. Sono conteggiati i file sorgente e le configurazioni che realizzano un comportamento, oltre alla documentazione quando questa costituisce un deliverable esplicito della Change Request. I test sono derivati dagli insiemi di impatto, ma non ne modificano la cardinalità; allo stesso modo sono esclusi gli artefatti generati automaticamente, come file di lock o output di compilazione.

Per i componenti non presenti prima della modifica viene utilizzato il percorso previsto per la loro introduzione. Questa scelta consente di includere nel *Starting Impact Set* anche nuovi moduli o componenti UI e di confrontare la previsione con la struttura effettivamente realizzata.

1.3 Insiemi di Impatto

Per ogni Change Request sono utilizzati gli insiemi riportati nella Tabella 1.1.

Tabella 1.1. Definizione degli insiemi utilizzati nell'Impact Analysis

Sigla	Denominazione	Significato
SIS	Starting Impact Set	Unità direttamente coinvolte dal requisito e necessarie per iniziare l'intervento.
CIS	Candidate Impact Set	SIS esteso con le unità potenzialmente raggiunte attraverso dipendenze statiche, contratti o configurazioni condivise.
AIS	Actual Impact Set	Unità effettivamente modificate per completare la Change Request.
FPIS	False Positive Impact Set	Unità previste nel CIS che non hanno richiesto alcuna modifica.
DIS	Discovered Impact Set	Unità modificate durante l'implementazione ma non incluse nella previsione iniziale.

Per costruzione, $SIS \subseteq CIS$. Gli insiemi di scostamento sono definiti come:

$$FPIS = CIS \setminus AIS \quad (1.1)$$

$$DIS = AIS \setminus CIS \quad (1.2)$$

1.4 Metriche di Accuratezza

La qualità della previsione è valutata tramite Precision e Recall:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} \quad (1.3)$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} \quad (1.4)$$

La Precision diminuisce quando il CIS contiene molte unità non modificate; la Recall diminuisce quando l'implementazione coinvolge unità non previste. Le due metriche devono essere interpretate congiuntamente: un CIS molto ristretto può apparire preciso ma omettere dipendenze rilevanti, mentre un CIS eccessivamente ampio può raggiungere Recall elevata senza offrire una selezione utile.

2 Modello Statico delle Dipendenze

La costruzione degli insiemi candidati parte dalle dipendenze statiche tra le unità di coordinamento del sistema e considera la direzione con cui una modifica può propagarsi. Le matrici riportano 0 sulla diagonale, 1 per una dipendenza diretta, 2 o più per una dipendenza indiretta e “-” quando non esiste un percorso nel modello considerato.

Il livello di astrazione è intenzionalmente architetturale: detection rule e classi estrattrici specialistiche, già descritte nel Reverse Engineering, sono aggregate dietro Inspector, poiché le Change Request non ne modificano la responsabilità.

2.1 Static Analysis Tool

La Tabella 2.1 sintetizza la raggiungibilità rilevante per CR1 e CR2. Le sigle indicano CLI (C), GUI (G), ProjectAnalyzer (PA), Inspector (I), FileUtils (FU), CallGraphExtractor (CGE) e CallGraphBuilder (CGB).

Tabella 2.1. Matrice di raggiungibilità del core di analisi

Da / A	C	G	PA	I	FU	CGE	CGB
C	0	-	1	2	2	3	2
G	-	0	1	2	2	3	2
PA	-	-	0	1	1	2	1
I	-	-	-	0	-	1	-
FU	-	-	-	-	0	-	-
CGE	-	-	-	-	-	0	-
CGB	-	-	-	-	-	-	0

CR1 interviene nel tratto PA-I-CGE e introduce CGB come aggregatore dei frammenti prodotti sui singoli file. CLI, GUI e FileUtils risultano candidati perché dipendono dall’orchestrazione o dalla gestione degli output, ma la natura opzionale del nuovo artefatto può evitare una loro modifica. CR2 parte invece dalla CLI e attraversa PA e FU; I e il sottosistema di report restano candidati solo qualora la propagazione delle opzioni richieda un cambiamento delle loro interfacce.

2.2 Flusso Statico della Web Application

Per CR4 il percorso principale attraversa frontend, gateway e servizio statico. La Tabella 2.2 utilizza le sigle Upload Project (UP), ProjectComponent (PC), Gateway (GW), Static Router (SR), Static Analysis utility (SA), Inspector (I), package Call Graph (CG) e CallGraphViewer (CGV).

Tabella 2.2. Matrice di raggiungibilità del flusso Web statico

Da / A	UP	PC	GW	SR	SA	I	CG	CGV
UP	0	1	1	2	3	4	4	2
PC	–	0	–	–	–	–	–	1
GW	–	–	0	1	2	3	3	–
SR	–	–	–	0	1	2	2	–
SA	–	–	–	–	0	1	1	–
I	–	–	–	–	–	0	–	–
CG	–	–	–	–	–	–	0	–
CGV	–	–	–	–	–	–	–	0

La matrice rende visibili due rami distinti: il ramo di richiesta, che termina nella costruzione di smell e grafo, e il ramo di presentazione, che conserva la risposta nel contesto del progetto e la inoltra al visualizzatore. La separazione è rilevante perché un cambiamento del contratto graph_data può richiedere modifiche su entrambi i rami anche quando la logica di detection non varia.

2.3 Configurazione dei Servizi

CR3 riguarda dipendenze di avvio che non emergono interamente dalle chiamate tra funzioni. La Tabella 2.3 completa quindi il modello con le relazioni di configurazione e packaging.

Tabella 2.3. Percorsi di propagazione infrastrutturale

Origine	Dipendenza	Propagazione possibile
docker-compose.yml	Build context, volumi, rete e variabili d’ambiente	Gateway e immagini dei servizi backend.
webapp/gateway/main.py	URL dei servizi	Router statico e router di report tramite chiamate HTTP.
Entry point del servizio statico	Import e registrazione del router	Utility di analisi, directory di output e package condivisi.
Entry point del servizio report	Import e registrazione del router	Generazione del report e relativa directory di lavoro.
Dockerfile dei servizi	Copia dei package e comando di avvio	Coerenza tra struttura locale e struttura interna al container.

2.4 Regole di Derivazione

Per ogni Change Request l’analisi segue lo stesso tracciato:

- (1) il SIS è ricavato dai deliverable e dai punti di ingresso direttamente interessati;
- (2) il CIS estende il SIS con le unità raggiungibili nelle matrici o legate da configurazioni e contratti condivisi;

- (3) l'AIS è determinato dalle unità che hanno richiesto una modifica effettiva, indipendentemente da dove questa fosse stata inizialmente prevista;
- (4) FPIS, DIS, Precision e Recall sono calcolati esclusivamente dopo aver fissato i tre insiemi precedenti.

Le dipendenze utilizzate per il CIS rappresentano un rischio di modifica, non un obbligo. Un componente raggiungibile può rimanere invariato quando l'interfaccia viene mantenuta retrocompatibile; in tal caso esso confluisce nel FPIS e resta comunque parte del successivo perimetro di regression testing.

3 Impact Analysis di CR1 e CR2

CR1 e CR2 condividono il core dello Static Analysis Tool ma introducono responsabilità differenti. CR1 costruisce il nuovo artefatto strutturale; CR2 espone e configura tale capacità dalla CLI. L’analisi viene pertanto mantenuta separata, evitando di attribuire a CR2 le modifiche già necessarie alla generazione del grafo.

3.1 CR1 – Generazione del Call Graph

3.1.1 *SIS e CIS*. Il punto iniziale comprende il nuovo package `call_graph` e i due componenti che coordinano l’ispezione e l’aggregazione dei progetti. La raggiungibilità mostrata nella Tabella 2.1 estende il perimetro a CLI, GUI e FileUtils: tutte consumano il flusso di `ProjectAnalyzer` o ne gestiscono l’output, ma non sono necessarie alla costruzione interna del grafo.

3.1.2 *AIS, FPIS e DIS*. L’implementazione ha isolato l’estrazione AST e la costruzione dell’artefatto nel nuovo package, collegandole a `Inspector` e `ProjectAnalyzer`. La modalità senza Call Graph mantiene il contratto precedente e non richiede modifiche ai consumer esterni. La Tabella 3.1 mostra l’appartenenza dei gruppi di unità ai cinque insiemi, evitando di ripetere gli stessi percorsi.

Tabella 3.1. Appartenenza delle unità agli insiemi di CR1

Gruppo di unità	N.	SIS	CIS	AIS	FPIS	DIS
<code>call_graph/__init__.py</code> ; <code>call_graph/call_graph_extractor.py</code> ; <code>call_graph/call_graph_builder.py</code> ; <code>components/inspector.py</code> ; <code>components/project_analyzer.py</code>	5	•	•	•	–	–
<code>cli/cli_runner.py</code> ; <code>gui/code_smell_detector_gui.py</code> ; <code>utils/file_utils.py</code>	3	–	•	–	•	–

Ne risultano $|SIS| = 5$, $|CIS| = 8$, $|AIS| = 5$, $|FPIS| = 3$ e $|DIS| = 0$. Le metriche sono:

$$Precision_{CR1} = \frac{5}{8} = 0,625 \approx 0,63 \tag{3.1}$$

$$Recall_{CR1} = \frac{5}{5} = 1,00 \tag{3.2}$$

La Recall completa indica che il confine necessario è stato previsto. La Precision inferiore all’unità deriva da una scelta prudenziale: CLI, GUI e FileUtils erano raggiungibili dall’orchestratore, ma parametri opzionali e valori predefiniti compatibili hanno evitato di modificarli in CR1. Essi rimangono comunque consumer da sottoporre a regressione.

3.2 CR2 – Estensione della CLI con Opzioni Avanzate

3.2.1 *SIS e CIS*. I quattro deliverable della CLI incidono sul parsing e sulla validazione degli argomenti, sulla propagazione verso `ProjectAnalyzer`, sulla selezione e scrittura dei file e sulla documentazione utente. `Inspector` e `ReportGenerator` sono inclusi nel CIS perché possono ricevere rispettivamente il comando di attivazione e il formato di output, ma la modifica non impone a priori un loro adattamento.

3.2.2 *AIS, FPIS e DIS*. Le opzioni vengono validate nella CLI e inoltrate all’orchestratore. La selezione dei percorsi e il merge nel formato richiesto sono assorbiti da FileUtils, mentre il README chiude il deliverable documentale. Il contratto di Inspector e il generatore di report pre-esistente non richiedono modifiche.

Tabella 3.2. Appartenenza delle unità agli insiemi di CR2

Gruppo di unità	N.	SIS	CIS	AIS	FPIS	DIS
cli/cli_runner.py; components/project_analyzer.py; utils/file_utils.py; README.md	4	•	•	•	–	–
components/inspector.py; report/report_generator.py	2	–	•	–	•	–

Ne risultano $|SIS| = 4$, $|CIS| = 6$, $|AIS| = 4$, $|FPIS| = 2$ e $|DIS| = 0$. Le metriche sono:

$$Precision_{CR2} = \frac{4}{6} \approx 0,67 \tag{3.3}$$

$$Recall_{CR2} = \frac{4}{4} = 1,00 \tag{3.4}$$

Anche per CR2 non emergono unità impreviste. I due falsi positivi mostrano che la propagazione termina prima dei componenti candidati: l’orchestratore converte le opzioni in parametri già compatibili con Inspector e delega la serializzazione alla utility esistente, senza modificare la semantica del reporting.

4 Impact Analysis di CR3 e CR4

CR3 e CR4 attraversano i confini della Web Application. La prima modifica il modo in cui i servizi vengono configurati e avviati; la seconda estende il contratto del flusso statico fino alla presentazione interattiva nel frontend. Il loro CIS comprende pertanto dipendenze non rappresentate dal solo Call Graph del codice, quali build context, schemi API e stato della UI.

4.1 CR3 – Portabilità tra Ambiente Locale e Docker

4.1.1 SIS e CIS. Il SIS comprende la configurazione di composizione, gli entry point che risolvono indirizzi e import dei servizi in scope e la documentazione di avvio. Il CIS aggiunge i router e le utility raggiunti dagli entry point, oltre ai Dockerfile che avrebbero potuto richiedere l'allineamento dei percorsi interni. Il componente AI-Based non è incluso nell'analisi funzionale.

4.1.2 AIS, FPIS e DIS. L'uniformazione ha richiesto default locali e override tramite variabili d'ambiente nel gateway, import portabili negli entry point e nei router, configurazione coerente delle directory e aggiornamento dei comandi di avvio. I Dockerfile già presenti non hanno richiesto modifiche; è invece emersa la necessità di controllare il contenuto inviato al build context mediante un file `.dockerignore` comune.

Tabella 4.1. Appartenenza delle unità agli insiemi di CR3

Gruppo di unità	N.	SIS	CIS	AIS	FPIS	DIS
<code>docker-compose.yml</code> ; <code>webapp/gateway/main.py</code> ; <code>webapp/services/staticanalysis/app/main.py</code> ; <code>webapp/services/report/app/main.py</code> ; <code>webapp/README.md</code>	5	•	•	•	–	–
<code>webapp/services/staticanalysis/app/routers/detect_smell.py</code> ; <code>webapp/services/staticanalysis/app/utis/static_analysis.py</code> ; <code>webapp/services/report/app/routers/report.py</code>	3	–	•	•	–	–
<code>webapp/gateway/Dockerfile</code> ; <code>webapp/services/staticanalysis/Dockerfile</code> ; <code>webapp/services/report/Dockerfile</code>	3	–	•	–	•	–
<code>.dockerignore</code>	1	–	–	•	–	•

Ne risultano $|SIS| = 5$, $|CIS| = 11$, $|AIS| = 9$, $|FPIS| = 3$ e $|DIS| = 1$. Le metriche sono:

$$Precision_{CR3} = \frac{8}{11} \approx 0,73 \quad (4.1)$$

$$Recall_{CR3} = \frac{8}{9} \approx 0,89 \quad (4.2)$$

Il CIS individua correttamente i punti di configurazione e la propagazione agli strati interni dei servizi. Lo scostamento è limitato alla strategia concreta di packaging: l'adeguamento è stato ottenuto senza modificare i tre Dockerfile, ma ha richiesto un file di esclusione non previsto. Il DIS evidenzia quindi un rischio specifico da verificare negli smoke test dei container.

4.2 CR4 – Visualizzazione del Call Graph nella WebApp

4.2.1 *SIS e CIS.* Il SIS segue i due rami della Tabella 2.2: generazione e trasporto di graph_data sul backend, conservazione e rendering sul frontend. Sono inclusi i nuovi componenti di visualizzazione, gli schemi del contratto, il tipo condiviso e le dipendenze frontend. Il CIS aggiunge il package Call Graph, Inspector e il provider del contesto, poiché una variazione della forma dei nodi o dello stato del progetto avrebbe potuto richiederne l’adattamento.

4.2.2 *AIS, FPIS e DIS.* Il contratto prodotto da CR1 è risultato sufficiente e non ha richiesto nuove modifiche a extractor e builder durante CR4. Il provider del contesto ha mantenuto la propria interfaccia; il nuovo stato è stato descritto nei tipi e popolato dalla pagina. Inspector ha invece richiesto l’allineamento tra nomi dei metodi e identificativi dei nodi. La gestione dei percorsi relativi e della selezione multipla ha inoltre coinvolto FileInput, non compreso nel CIS iniziale.

Tabella 4.2. Appartenenza delle unità agli insiemi di CR4

Gruppo di unità	N.	SIS	CIS	AIS	FPIS	DIS
webapp/app/upload-project/page.tsx; webapp/components/ProjectComponent.tsx; webapp/components/CallGraphViewer.tsx; webapp/components/GraphFilters.tsx; webapp/types/types.ts; webapp/gateway/main.py; webapp/services/staticanalysis/app/routers/detect_smell.py; webapp/services/staticanalysis/app/schemas/requests.py; webapp/services/staticanalysis/app/schemas/responses.py; webapp/services/staticanalysis/app/utils/static_analysis.py; webapp/package.json	11	•	•	•	–	–
components/inspector.py	1	–	•	•	–	–
call_graph/call_graph_extractor.py; call_graph/call_graph_builder.py; webapp/context/ProjectContext.tsx	3	–	•	–	•	–
webapp/components/Project/FileInput.tsx	1	–	–	•	–	•

Ne risultano $|SIS| = 11$, $|CIS| = 15$, $|AIS| = 13$, $|FPIS| = 3$ e $|DIS| = 1$. Le metriche sono:

$$Precision_{CR4} = \frac{12}{15} = 0,80 \tag{4.3}$$

$$Recall_{CR4} = \frac{12}{13} \approx 0,92 \tag{4.4}$$

CR4 presenta la Precision più alta tra le quattro analisi, poiché il flusso full-stack e il contratto dati sono stati delimitati correttamente. Il DIS mostra tuttavia che la visualizzazione dipende anche dal modo in cui file e percorsi vengono raccolti dal browser, non soltanto dal rendering.

4.3 Valutazione Complessiva

La Tabella 4.3 riassume gli scostamenti tra previsione e implementazione. Le cardinalità sono conteggiate per Change Request; una stessa unità può quindi comparire in più righe quando svolge responsabilità diverse in interventi successivi.

Tabella 4.3. Sintesi degli insiemi e delle metriche

CR	SIS	CIS	AIS	FPIS	DIS	Precision	Recall
CR1	5	8	5	3	0	0,63	1,00
CR2	4	6	4	2	0	0,67	1,00
CR3	5	11	9	3	1	0,73	0,89
CR4	11	15	13	3	1	0,80	0,92

Considerando congiuntamente le quattro osservazioni, 29 unità effettivamente impattate erano comprese nelle 40 candidature, mentre le occorrenze di impatto effettivo sono 31. Ne derivano una Precision aggregata pari a $29/40 \approx 0,73$ e una Recall aggregata pari a $29/31 \approx 0,94$. L’analisi ha quindi privilegiato una copertura elevata, mantenendo un numero contenuto di falsi positivi.

I due DIS appartengono a livelli differenti: la configurazione del build context per CR3 e l’acquisizione dei file nel frontend per CR4. La loro presenza evidenzia scostamenti circoscritti tra l’impatto previsto e quello effettivamente riscontrato, mentre i FPIS rappresentano dipendenze considerate in via cautelativa ma non interessate dalle modifiche. Nel complesso, i risultati mostrano un’elevata copertura degli impatti effettivi e un numero contenuto di falsi positivi, confermando l’adeguatezza del modello di analisi adottato.